

Домашнее задание 1.

Перевод открытой выгрузки в воспроизводимую Lakehouse-таблицу

Вы получаете открытую выгрузку от внешнего источника. Аналитику нужен SQL, Data Engineer хочет воспроизводимую загрузку, а другой пользователь — понятную схему и историю версий. Нужно превратить выгрузку в таблицу, которую одинаково видят Spark и Trino. Используйте стенд и разобранные шаги семинара: новые обязательные сервисы не нужны.

Сначала подготовьте среду и повторите учебный пример, который продемонстрировал преподаватель на первом семинаре «Настройка S3 и колоночные форматы: подготовка источника данных», затем работайте со своим источником. На показе преподаватель использует DataGrip как клиент Trino, вы можете использовать DataGrip или CLI, оценка от выбора клиента не зависит:

источник → raw object → Parquet → Iceberg snapshot → SQL → вывод

Где взять инфраструктуру

Скачайте `lecture_01_student.zip` из папки материалов и распакуйте. Внутри `lecture_01_student/` лежит условие, инструкции и самостоятельная папка `infra/`. Начните с `README.md` комплекта.

Формат

- Работа индивидуальная: каждый студент готовит собственное решение.
- Среда: локальный стенд из `infra/` на macOS или Windows.
- Данные: выбранная вами открытая выгрузка; синтетический `events_demo.csv` нужен только для повторения учебного примера.
- Оценка этого ДЗ: от 0 до 10 баллов по критериям ниже.

Результат

1. Зафиксировать происхождение и ограничения открытого набора.
2. Сохранить неизменённый raw-объект в S3-совместимом хранилище.
3. Задать явную схему и преобразовать данные в Parquet.
4. Создать Iceberg-таблицу и подтвердить наличие snapshot.
5. Прочитать таблицу Spark и Trino.
6. Выполнить содержательный аналитический и федеративный SQL.
7. Объяснить измеренный эффект, не выдавая один запуск за универсальный закон.

Выбор датасета

Минимальные требования:

- открытая лицензия или ясные условия использования;
- нет персональных, секретных и требующих авторизации данных;
- ориентир не меньше 100 000 строк после подготовки; меньший содержательный набор можно

согласовать с преподавателем;

– не меньше трёх осмысленных полей для вопроса: дата/время, категория, числовая мера; не добавлять искусственные колонки ради количества;

– источник можно повторно получить по ссылке или команде;

– выбирайте ограниченный период или часть выгрузки, которая помещается на ноутбуке; скачивать весь архив источника не нужно. Зафиксируйте границы выборки в отчёте. Заданный коэффициент сжатия или ускорения не требуется.

Подходящие направления:

– [Wikimedia Pageviews](#) — почасовые просмотры, данные CC0;

– [Open-Meteo Historical Weather API](#) — временные ряды погоды, условия CC BY 4.0 указаны в спецификации;

– [NYC TLC Trip Record Data](#) — поездки такси; проверьте формат выбранного периода и условия использования.

Можно выбрать другой набор после короткого согласования с преподавателем. Не использовать GitHub public events API как источник большого исторического набора: официальный endpoint ограничен недавними событиями и объёмом ответа.

Исследовательский вопрос

Обязательный результат: до преобразования сформулируйте один исследовательский вопрос, на который ответит SQL.

Примеры:

– «В какие часы выбранные страницы Wikimedia получают больше просмотров?»

– «Как распределяется число поездок и стоимость по дням недели?»

– «Какие города имеют похожий суточный профиль температуры?»

Плохой вопрос:

«Сколько строк в таблице?» — это проверка, а не анализ.

Задание

0. Среда и учебный пример

Сначала установите и проверьте среду. Пройдите [инструкцию macOS](#) или [инструкцию Windows](#). Минимальный результат: сервисы доступны, в MinIO есть бакеты `raw` и `datalake`, Trino показывает каталоги `lakehouse` и `memory`. Создайте папку `evidence/` в каталоге своей сдачи и сохраните туда вывод `docker compose ps --all` как `compose-ps.txt`.

Затем повторите показанный преподавателем путь. Все команды ниже выполняются из `infra/` распакованного комплекта `lecture_01_student/`: на macOS — в Terminal, на Windows — в Ubuntu/WSL. Docker Desktop и сервисы должны быть запущены.

```
python3 generate_dataset.py --rows 200000 --out data/events_demo.csv
wc -l data/events_demo.csv
ls -lh data/events_demo.csv
docker compose exec spark spark-submit /scripts/01_upload_csv.py
/data/events_demo.csv docker compose exec spark spark-submit
/scripts/02_csv_to_parquet.py
```

```
docker compose exec spark spark-submit /scripts/03_iceberg.py
```

После Spark-шагов подключите DataGrip по [инструкции](#) и выполните блоки [infra/trino/scripts/seminar_demo.sql](#) по порядку. Выберите подключение Trino для SQL-файла. Файл содержит контрольные запросы, историю и создание учебного справочника. Если используете CLI, тот же сценарий выполняется из [infra/](#):

```
docker compose exec -T trino trino < trino/scripts/seminar_demo.sql
```

После каждой команды прочитайте результат; при ошибке не переходите к следующему шагу. Для файла ожидаются 200 001 строка с заголовком, для итоговой таблицы — 200 000 записей.

Проверьте, что:

- в MinIO появился объект [raw/events/2026/events.csv](#); проверяйте ключ и размер — preview CSV, JSON и Parquet в установленной консоли не поддерживается;
- после записи Parquet число строк сохранилось;
- SQL выводит два новых snapshots Iceberg после начальной загрузки и добавления второй порции, а также результат JOIN каталогов [lakehouse](#) и [memory](#) (при повторном запуске примера история snapshots может быть длиннее);
- строки без соответствия в справочнике попадают в группу «Не сопоставлен».

Учебный CSV синтетический: его события не описывают реальных людей, связанные покупки или пользовательскую воронку. Сумму [price_rub](#) нельзя называть реальной выручкой, а отношение числа событий [purchase](#) к [page_view](#) — пользовательской конверсией. Поля и ограничения разобраны в [паспорте учебного датасета](#). В ДЗ используйте смысл и ограничения своего источника.

Учебные скрипты используют фиксированные пути [events](#) и перезаписывают результаты своего примера. Для своей работы задайте отдельные пути и таблицы из следующих этапов. Хостовый каталог [infra/scripts/](#) доступен контейнеру Spark как [/scripts/](#), а [infra/data/](#) — как [/data/](#). Например, ваш [pipeline.py](#) можно разместить в [infra/scripts/hw01/](#) и запускать из [infra/](#) так:

```
docker compose exec spark spark-submit /scripts/hw01/pipeline.py
```

Перед запуском задайте в своём коде пути собственного датасета. Ниже [s3://](#) обозначает адрес объекта; для чтения и записи файлов средствами Spark используйте [s3a://](#), как в учебном скрипте. Для [<student>](#) и [<dataset>](#) выберите SQL-имена без дефисов и пробелов, например [student_01](#) и [weather](#).

Итоговые файлы сдавайте в структуре из раздела «Что сдавать», без учебного CSV. Если среда не запустилась, приложите к обращению преподавателю ОС, объём RAM, неудавшуюся команду и текст ошибки; команды диагностики есть в инструкции вашей ОС.

1. Паспорт источника

Создайте [report.md](#) и укажите:

- название и официальную ссылку;
- дату получения;
- лицензию/условия;
- исходный формат и размер;
- единицу наблюдения: что означает одна строка;
- выбранный исследовательский вопрос;

– известные ограничения или пропуски.

2. Raw без скрытых изменений

Сохраните исходник в бакет `raw` с понятным key, например:

```
s3://raw/<student>/<dataset>/ingestion_date=2026-09-10/source.csv
```

В отчёте приложите команду получения данных и вывод листинга объекта с размером. Raw не редактировать вручную. Если источник отдаёт JSON, Parquet или другой формат, сохраните оригинал, затем отдельным воспроизводимым шагом подготовьте табличный CSV. Так путь CSV → Parquet из программы сохраняется и для NYC TLC, где исходник может уже быть Parquet. Этот дополнительный шаг нужен для учебного сравнения, а не как рекомендация всегда превращать Parquet в CSV.

3. Явная схема и DQ

Создайте `pipeline.py`. Не используйте `inferSchema=True` в итоговом решении: задайте типы явно.

Обязательные проверки:

- число строк больше нуля;
- ключевые столбцы существуют;
- временной столбец парсится;
- числовая мера не содержит невозможных значений;
- доля `NULL` в выбранном ключевом поле рассчитана;
- до и после преобразования число строк совпадает либо расхождение объяснено.

Если отбрасываете строки, сохраните исходные значения и причину в отдельный `reject`-путь с колонкой `reject_reason`. Сверьте `raw = accepted + rejected`. Если отклонённых строк нет, достаточно воспроизводимой проверки с нулевым результатом; искусственные ошибки добавлять не нужно. Правила зависят от смысла данных: отрицательная температура допустима. Пример явных типов есть в конспекте семинара; распространите его на все сохраняемые поля своего источника.

4. Parquet и измерение

Запишите очищенные данные в:

```
s3://datalake/<student>/<dataset>/parquet/
```

Выберите разумное партиционирование и объясните его. Не партиционируйте по уникальному идентификатору.

Измерьте:

Метрика	Подготовленный CSV	Parquet	Как измеряли
Размер, МиБ (байты / 1024 ²)			
Число строк			
Время одного аналитического запроса, с			
Поля, используемые запросом			

Сравнивайте одинаковые принятые строки, типы и один запрос. Размер исходного raw укажите отдельно, если он отличается от подготовленного CSV. Задайте схему до начала таймера; замеряйте выполнение запроса, а не создание ленивого DataFrame. Первый запуск запишите отдельно, затем сделайте минимум два повторения для каждого формата и покажите все времена. Укажите состояние кэша и ограничения ноутбука. За образец возьмите порядок измерения в [infra/scripts/02_csv_to_parquet.py](#), заменив схему и аналитический запрос на подходящие вашему источнику. Число полей запроса не равно фактически прочитанным байтам.

5. Iceberg

Создайте таблицу:

```
lakehouse.<student>.<dataset>
```

Сделайте минимум две успешные записи данных:

1. начальная загрузка;
2. добавление другой, непустой и непересекающейся порции исходника.

Например, заранее разделите исходник на два диапазона дат. Приложите условия разделения и число строк в обеих порциях; итог должен совпасть с полным принятым набором. Изменение только схемы не заменяет второй шаг: оно может обновить метаданные без нового snapshot данных.

Пример двух записей из непересекающихся порций разобран на семинаре.

Покажите metadata table `$snapshots` и объясните, чем отличаются snapshots. В Trino имя с суффиксом заключается в двойные кавычки: `lakehouse.<student>."<dataset>$snapshots"`.

Недостаточно приложить скриншот: добавьте SQL и текстовый вывод.

6. Spark и Trino видят одну таблицу

Приложите:

- один Spark-запрос к Iceberg-таблице;
- один эквивалентный Trino-запрос;
- сверку ключевого результата (например, число строк или сумма меры).

В DataGrip назначьте файлу `queries.sql` своё подключение Trino. DataGrip отправляет запрос серверу; хранение и обработку выполняют компоненты стенда. Для DECIMAL заранее согласуйте точность: Spark и Trino могут возвращать AVG с разным scale. В учебном запросе используется `avg(CAST(price_rub AS decimal(18,6)))`, чтобы получить шесть знаков. Для своего источника подберите тип без потери значимых разрядов.

7. Федеративный SQL

Создайте небольшой справочник в `memory` catalog Trino либо используйте другой согласованный

каталог. Выполните JOIN с Iceberg-таблицей и ответьте на исследовательский вопрос.

Объясните происхождение и правило справочника. Убедитесь, что ключи не дублируются, а неполное покрытие не теряет строки незаметно: используйте `LEFT JOIN` с явной группой «Не сопоставлен» либо обоснуйте фильтрацию. `memory` — учебный непостоянный источник; его содержимое нужно восстанавливать командами из `queries.sql`.

Поясните:

- какие каталоги участвуют;
- где физически находятся исходные данные;
- что делает Trino во время запроса;
- почему это называется федерацией.

8. Архитектурный вывод

В 5–8 предложениях ответьте:

1. Что дал S3-слой?
2. Что дал Parquet?
3. Что добавил Iceberg?
4. Зачем понадобились и Spark, и Trino?
5. Какое самое слабое место текущего локального решения?

Что нужно сдать

```
student-XX-hw01/
├── README.md # как запустить с чистого стенда
├── report.md # источник, таблицы измерений, вывод
├── pipeline.py # raw → validation → parquet → iceberg
├── queries.sql # проверочные, аналитический и федеративный SQL
├── schema.md # поля, типы, смысл, nullable
├── evidence/
│   ├── compose-ps.txt
│   ├── object-listing.txt
│   ├── spark-result.txt
│   └── trino-result.txt
```

Не сдавайте сам большой датасет и Docker volumes. README должен позволять преподавателю повторить работу.

Оценка: максимум 10 баллов

Каждый из десяти пунктов ниже даёт **1 балл**, если результат получен и подтверждён кодом/SQL и сохранённым выводом.

Раздел	Баллы	За что начисляется каждый балл
Источник и вопрос	1	1: источник можно получить повторно, указаны условия, единица наблюдения и содержательный вопрос
Raw, схема и качество	2	1: оригинал сохранён в S3, задана явная схема; 1: выполнены проверки и объяснены допустимые NULL, ошибки и принятые/отклонённые строки
Parquet и измерения	2	1: Parquet читается, число принятых строк и типы сверены, выбор партиций объяснён; 1: сравнение одинаковых данных и запросов выполнено корректно, приведены все времена и вывод
Iceberg и история	2	1: выполнены начальная загрузка и добавление непересекающейся порции, итоговые количества/ключи проверены; 1: показаны соответствующие snapshots и объяснена связь каталога, метаданных и файлов
SQL и результат	2	1: эквивалентные Spark/Trino-запросы дали согласованный результат; 1: JOIN двух каталогов отвечает на вопрос, ключи и сохранность строк проверены
Воспроизводимость и объяснение	1	1: README позволяет повторить работу, приложены результаты и объяснено назначение каждого слоя и ограничение решения
Итого	10	Десять проверяемых пунктов по одному баллу

Например, корректный Parquet без корректного измерения времени даёт 1 из 2 за раздел. Два snapshots при дублирующейся второй порции не дают балл за правильное добавление данных. Поднятый Docker и повторение синтетического примера — подготовка к работе со своим источником, отдельных баллов за них нет.

Ускорение и заданный коэффициент сжатия **не требуются**. Честно измеренное замедление с корректным объяснением может получить полный балл. Качество оценивается по смыслу и проверкам, а не по эффективности числа или выбору DataGrip/CLI.

Это оценка конкретного ДЗ. Его вес в оценке курса определяется отдельно в LMS.

Time travel (чтение старой версии через [FOR VERSION AS OF](#)) — необязательный материал преподавателя; отдельного балла или нового обязательного пункта за него нет.

Ограничения безопасности

- Стенд учебный и локальный; не выставлять порты MinIO, PostgreSQL и Trino в интернет.
- Не коммитить реальные токены и пароли внешних сервисов.
- Не использовать данные с персональными идентификаторами.
- Перед публикацией репозитория проверить историю на секреты и большие файлы.

Необязательное дополнение

Сделайте один понятный график по результату SQL и подпишите один вывод. График должен отвечать на исследовательский вопрос, а не быть декоративным. Дополнительный продукт или новый движок не требуется. График не даёт баллов сверх 10 и не заменяет обязательные пункты критериев.

Быстрая самопроверка

- После семинара среда установлена и учебный пример воспроизведён.
- Источник можно получить заново.
- Raw-объект сохранён без ручной правки.
- Схема задана явно.
- Проверки качества воспроизводимы.
- Число строк сверено.
- Есть минимум два Iceberg snapshots.
- Spark и Trino дают согласованный результат.
- Федеративный JOIN использует два каталога.
- Таблица измерений заполнена фактическими данными.
- Вывод отвечает на исходный вопрос.